RECURSO DE PROFUNDIZACIÓN – Clase 10

Frameworks Modernos de Frontend: Arquitectura, Componentes y Paquetes

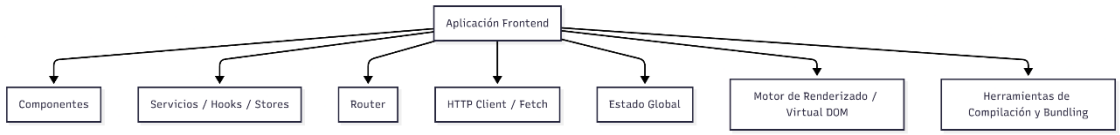
Este recurso extiende la teoría revisada en clase, permitiendo comprender cómo están compuestos internamente Angular, React y Vue, cuáles son sus partes fundamentales, cómo se organizan sus proyectos y qué patrones comparten entre ellos.

1. Arquitectura general de los frameworks modernos

Aunque Angular, React y Vue son diferentes en enfoque, siguen un patrón de arquitectura común:

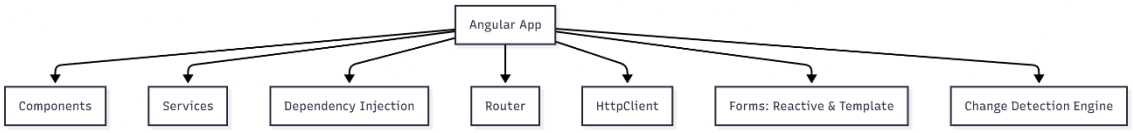
1. **Capa de Componentes (UI)**
2. **Capa de Lógica (Servicios/Stores/Hooks)**
3. **Capa de Integración (HTTP/API)**
4. **Capa de Navegación (Ruteo)**
5. **Capa de Estado Global**
6. **Capa de Construcción (Build/Bundling)**

1.1 Diagrama general de paquetes en frameworks modernos

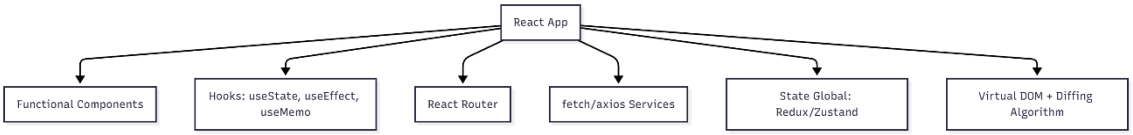


2. Angular vs React vs Vue: Diagrama de Paquetes Internos

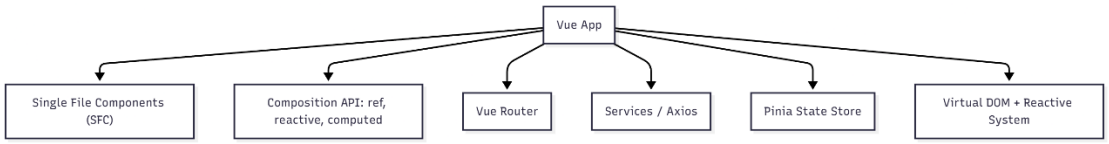
2.1 Angular (Standalone Architecture)



2.2 React (Functional + Hooks Architecture)



2.3 Vue (Composition API Architecture)



3. Comparación profunda de capas internas

Tabla 1. Estructura de componentes

Elemento	Angular	React	Vue
Vista	Template HTML	JSX	Template HTML
Lógica	Clase TS	Función JS/TS	Script setup / Composition API
Estilos	CSS/SCSS propios	CSS-in-JS / CSS	<style scoped>
Encapsulación	Shadow-like	Por componente	Scoped CSS

Tabla 2. Servicios / Lógica compartida

Angular	React	Vue
Servicios con DI	Custom hooks	Composition functions
Injectable	use...()	use...()
Singleton implícito	No definido	Pinia / composición

Tabla 3. Inyección de dependencias

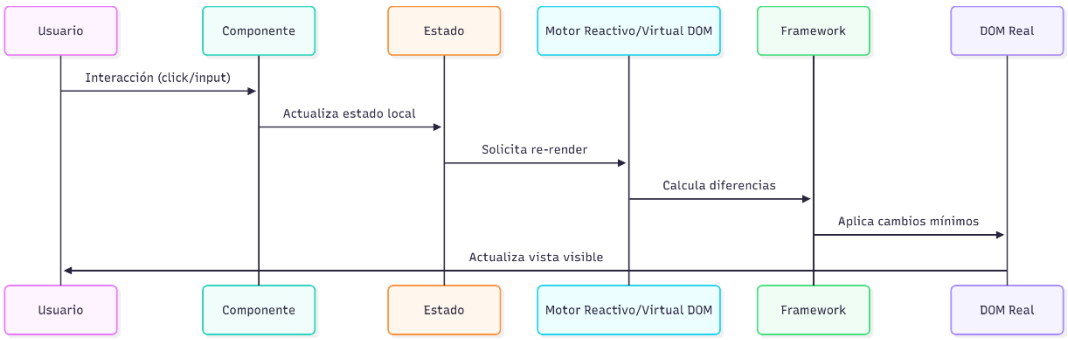
Framework	Uso
Angular	DI nativa con decoradores, jerarquía de inyectores
React	Context API (no DI real)
Vue	Provide/Inject API

Tabla 4. Manejo de Estado Global

Framework	Mecanismo
Angular	Servicios + NGRX (opcional)
React	Redux Toolkit, Zustand, Context API
Vue	Pinia (oficial), Vuex (legacy)

4. Diagrama comparativo de flujo interno

Este diagrama unifica el procesamiento interno de los 3 frameworks:



5. Ciclo de vida comparado

Etapa	Angular	React	Vue
Inicialización	ngOnInit()	useEffect(..., [])	onMounted()
Actualización	ngOnChanges()	useEffect()	onUpdated()
Limpieza	ngOnDestroy()	return cleanup en useEffect()	onUnmounted()

6. Estructuras de proyecto modernas

6.1 Angular

```
src/
 app/
  features/
  services/
  models/
  app.routes.ts
  app.component.ts
 main.ts
 app.config.ts
```

6.2 React

```
src/
 features/
 components/
 services/
 models/
 App.tsx
 main.tsx
```

6.3 Vue

```
src/
 features/
 components/
 services/
 models/
 router/
 App.vue
 main.ts
```

7. Patrón “Capa de Servicios”

Todos los frameworks modernos usan el mismo patrón arquitectónico, con diferencias en sintaxis:

7.1 Angular (HttpClient)

```
listar() {
  return this.http.get<Producto[]>(this.url);
}
```

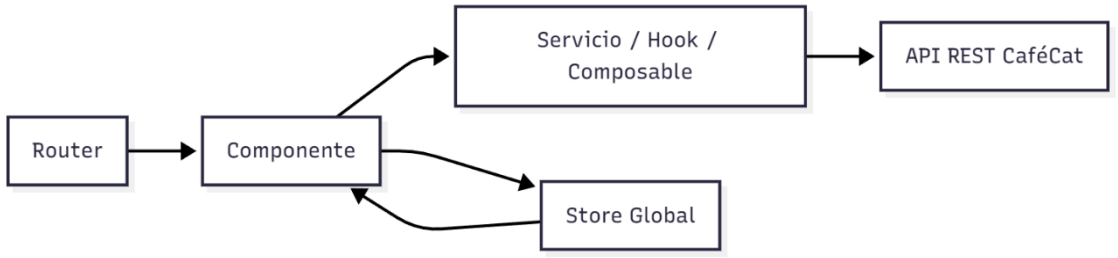
7.2 React (fetch/axios)

```
export async function listarProductos() {
  const res = await fetch(url);
  return res.json();
}
```

7.3 Vue (fetch/axios + Composition API)

```
export async function useProductos() {
  const res = await fetch(url);
  return await res.json();
}
```

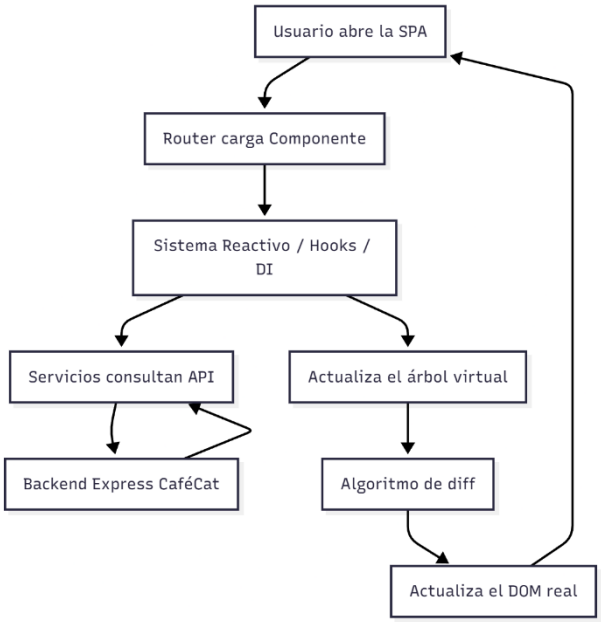
8. Diagrama general de relación: Componentes – Servicios – Router – Estado



9. Bundle, Compilación y Despliegue

Etapa	Angular	React	Vue
Build	ng build	npm run build	npm run build
Motor	esbuild + Angular Compiler	Vite + Rollup	Vite + Rollup
Salida	dist/	dist/	dist/
Deploy	Firebase, Nginx, Vercel	Vercel, Netlify	Vercel, Netlify

10. Diagrama final unificado: “Cómo funciona una SPA moderna”



Conclusiones del Recurso de Profundización

Este módulo permite al estudiante comprender:

- Cómo se estructura internamente un framework moderno.
- Diferencias arquitectónicas entre Angular, React y Vue.
- El rol del Virtual DOM, sistema reactivo e inyección de dependencias.
- Cómo los 3 frameworks se alinean en un patrón común:
  Componentes → Servicios → Estado → Renderizado → Router → API.
- Por qué los frameworks modernos han convergido hacia arquitecturas similares.